

AgenticPD: A Stage-Aware Agentic Framework for Physical Design QoR Optimization

Shuo Ren Zijin Cheng Yaohui Han Libo Shen Leilei Jin
Wanting Tian Rongliang Fu Chao Wang Bei Yu Tsung-Yi Ho

Abstract

Physical design quality-of-results (QoR) optimization is hard and expensive. Choices made at one stage can help or hurt later stages. Each evaluation requires a costly EDA run through the full flow. While existing methods still treat optimization as flat parameter tuning or a LLM-based script generation task, we present AgenticPD, a stage-aware agentic framework for physical design QoR optimization. Instead of re-running the full flow after every trial, AgenticPD is organized around the stage boundaries of the physical design flow, where a Judge Agent navigates the search and stage-specialized agents make local decisions within their own stage using stage-local tools. Additionally, the agent harness in AgenticPD provides structured observations, execution history, and agent context management. As a result, the system can branch from prior intermediate states and reuse checkpoints to continue the optimization procedure, and every candidate is evaluated at the post-route signoff. Across these baselines, AgenticPD achieves strong post-route timing while remaining competitive in power and area.

Keywords

Physical Design, QoR Optimization, Agentic EDA

1 Introduction

As modern chips grow in design complexity, achieving strong performance, power, and area (PPA) under tight design schedules has become increasingly difficult [11]. Physical design (PD) plays a central role in this process [6], since implementation decisions made during backend optimization directly affect final chip quality. Moreover, physical design quality-of-results (QoR) optimization is far from straightforward because the implementation flow spans multiple connected stages, including floorplan, placement, clock tree synthesis, and routing [13]. Each stage has its own local objectives, optimization methods, and feedback metrics. Yet stage-local improvements do not always translate into better final QoR. A decision that appears highly effective at one stage may leave too little optimization space for later stages and can ultimately degrade final performance, power, or area. As a result, physical-design optimization must be understood not only at the level of individual stages, but also in terms of how decisions propagate across the full flow. Together, these cross-stage dependencies make QoR optimization a challenging full-flow problem [13].

Existing methods for physical design QoR optimization can be broadly divided into two categories. The first category includes black-box tuning methods, such as AutoTuner [12], PPATuner [7], REMOTune [26], and FastTuner [10], which improve sampling efficiency by automatically exploring the configuration space. However, these methods typically treat the physical design flow as a flat input-output mapping and do not explicitly exploit stage-wise structure, intermediate feedback, or reusable checkpoint state.

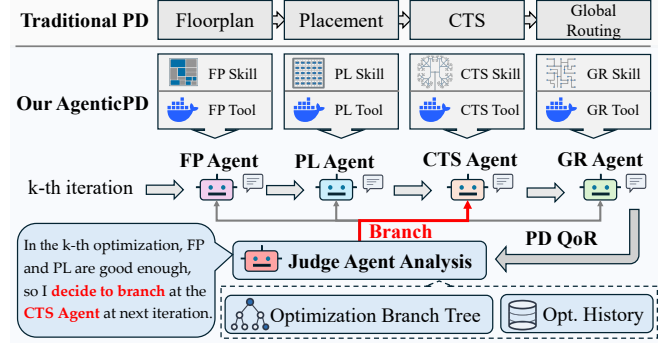


Figure 1: AgenticPD uses a multi-agent system to optimize the physical design flow instead of traditional uninterpretable black-box decisions.

The second category includes recent LLM-based systems, such as ORFS-Agent [8], OpenROAD-Assistant [20], OpenROAD Agent [24], and ChatEDA [25], which rely on language models for script generation and document exploration. However, these systems are still largely script-generation-like in how they organize optimization. Such a formulation is not well suited to physical-design QoR optimization, because this task is multi-stage, state-dependent, and requires decisions to be revised according to real execution feedback. As a result, neither category fully addresses PD QoR optimization as a structured multi-stage decision problem.

To address these challenges, we present AgenticPD, a stage-aware agentic framework for physical design QoR optimization. As shown in Fig. 1, AgenticPD organizes optimization around the structure of the full physical design flow and supports more effective exploration of the QoR search space, rather than treating each trial as an independent attempt. This can lead to stronger optimization outcomes than flat-trial baselines. Overall, the main contributions are summarized as follows:

- AgenticPD organizes physical design QoR optimization around stage boundaries rather than flat trials, enabling checkpoint-based branching and non-sequential exploration over the physical design optimization flow.
- We formulate physical-design QoR optimization as a structured multi-stage decision problem with reusable intermediate states, enabling each iteration to target specific stages without re-executing the entire flow.
- We design a Harness Skill and stage-level PD Skills that expose structured observations and execution history to the agents.
- Experiments show that, in our evaluation, AgenticPD achieves the best post-route timing among the compared methods, both vanilla LLM baselines and prior SOTA tuners, while keeping power and area competitive.

2 Preliminaries and Motivations

2.1 Physical Design

In physical design flow [13], we focus on four controllable stages: floorplanning (FP), placement (PL), clock tree synthesis (CTS), and routing (RT). Floorplan determines the coarse layout and resource budget of the design. Placement arranges cells under timing and density constraints. CTS builds the clock network to control clock latency and skew. Routing establishes the physical interconnects between placed cells: it first plans coarse interconnect paths (global routing), then assigns exact tracks and vias to produce the final routed layout (detailed routing). AgenticPD treats these two steps as one routing stage. Every candidate is carried through detailed routing, and all in-loop feedback and all reported metrics are measured at the post-route signoff. This makes each iteration more expensive, but the optimizer always observes the true final-layout quality, and no intermediate result is ever used as a proxy for it. We formulate the physical design flow as an ordered sequence of stages:

$$\mathcal{S} = (\text{FP}, \text{PL}, \text{CTS}, \text{RT}), \quad (1)$$

where $s \in \mathcal{S}$ denotes a physical design stage. Each stage s has its own action space Θ_s , representing the tunable parameters at that stage. The full action space factorizes as

$$\Theta_{\text{PD}} = \Theta_{\text{FP}} \times \Theta_{\text{PL}} \times \Theta_{\text{CTS}} \times \Theta_{\text{RT}}. \quad (2)$$

Because the stages execute in a fixed order, any selected stage $b \in \mathcal{S}$ partitions the flow into two parts: the stages before b , denoted $\text{Bef}(b)$, and the stages after b , denoted $\text{Aft}(b)$. For example, $\text{Bef}(\text{CTS}) = \{\text{FP}, \text{PL}\}$ and $\text{Aft}(\text{CTS}) = \{\text{RT}\}$.

Moreover, a single complete flow execution is specified by an action tuple

$$\mathbf{a} = (a(\text{FP}), a(\text{PL}), a(\text{CTS}), a(\text{RT})) \in \Theta_{\text{PD}}, \quad (3)$$

where $a(s) \in \Theta_s$ denotes the action applied at stage s . Executing $\mathbf{a}$ runs the four stages in order and produces a stage-level QoR tuple $Q(s) = (Q_1(s), \dots, Q_m(s))$ after each stage s , where each dimension corresponds to a design metric such as worst negative slack (WNS), total negative slack (TNS), power, or area. Because the routing stage runs through detailed routing, the RT-stage result of a completed flow is the post-route QoR of the whole candidate, denoted $Q(\mathbf{a})$; this is the quantity the optimizer observes in the loop and the quantity we report.

In practice, physical design optimization is not a one-shot problem. Given a post-synthesis netlist D (the input to the PD flow) and an iteration budget N , the optimizer executes a sequence of N complete flows:

$$\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_N, \quad \mathbf{a}_k \in \Theta_{\text{PD}}, \quad (4)$$

where each $\mathbf{a}_k$ is one complete action tuple decided during the optimization process. The in-loop objective is to optimize the best post-route QoR found within the budget:

$$\max_{k \in \{1, \dots, N\}} Q_k, \quad (5)$$

where $Q_k = Q(\mathbf{a}_k)$ is the post-route QoR of the k -th iteration. The reported result is simply the best candidate found, $\mathbf{a}^* = \arg \max_k Q_k$, whose QoR is already measured at the signoff, so no separate final evaluation step is needed.

2.2 Agentic AI and Agentic EDA

Recent progress in Agentic AI shows that language models can solve complex tasks more effectively when they are allowed to reason, call tools, inspect execution results, and revise decisions over multiple steps rather than produce one static output [2, 5, 14, 15, 18]. This interaction pattern is especially useful when task state evolves during execution and when later actions depend on earlier observations.

The EDA community has also started to adopt this paradigm and apply agentic AI to EDA problems. Cadence introduced ChipStack for AI-assisted chip design and verification [3]. Synopsys introduced AgentEngineer for multi-agent automation of RTL and verification tasks [22]. Siemens EDA launched Fuse EDA AI Agent for multi-tool automation from design to physical implementation [21]. However, these agentic EDA tools still mainly focus on design, verification, and workflow automation, while the use of agentic AI for physical design optimization remains underexplored.

2.3 Motivations

Physical-design QoR optimization is a natural fit for an agentic framework for three reasons. (1) It is not a one-shot prediction but a sequential process across FP, PL, CTS, and routing, where each stage has its own objectives, action space, and feedback yet depends on the state produced by earlier stages—closer to multi-step decision making than to flat parameter generation. (2) It requires both prior knowledge and execution feedback: prior knowledge guides stage-local choices at the start, but the true effect of a decision is revealed only after running the tools, so the optimizer must often revise or discard its initial judgment. (3) It requires repeated decision making and adjustment driven by execution feedback, which a one-shot script cannot naturally support. **Taken together, these properties make physical-design QoR optimization a natural fit for an agentic, stage-aware framework.**

3 Methodology

3.1 Optimization Tree

We organize the full optimization history as a rooted tree $\mathcal{T}$. The root n_0 represents the post-synthesis design D before any PD stage runs. Each time a stage s is executed in iteration k , a node is created:

$$n_k^s = (a_k(s), Q_k(s)), \quad (6)$$

where $a_k(s) \in \Theta_s$ is the action taken at stage s and $Q_k(s)$ is the stage-level QoR observed after execution. Every root-to-leaf path traverses all four stages in order and corresponds to one complete PD flow with action tuple $\mathbf{a}_k$ (Equation (3)).

Branching. Not every iteration needs to start from scratch. If a previous iteration already produced a good result at some early stage, re-running that stage with a different action is wasteful; the optimizer should focus its budget on the stages that still have room to improve. Branching makes this possible. At iteration k , the optimizer selects an intermediate node $\hat{n}$ from a previously completed path at some stage $b \in \mathcal{S}$ and starts a new branch from there. All stages before b are inherited from the ancestor path leading to $\hat{n}$ —the $\text{Bef}(b)$ results are reused at zero cost—and only stage b itself and the stages in $\text{Aft}(b)$ are re-executed with new actions. The resulting new nodes

$\{n_k^s\}_{s \in \{b\} \cup \text{Aft}(b)}$ are attached as a subtree under $\hat{n}$:

$$\mathcal{T}_k = \mathcal{T}_{k-1} \cup \{n_k^s\}_{s \in \{b_k\} \cup \text{Aft}(b_k)}, \quad (7)$$

where b_k is the branching stage chosen for iteration k . When $b_k = \text{FP}$ (the first stage), branching from the root reduces to running a completely new flow from scratch. This tree structure turns the flat iteration sequence (Equation (4)) into a structured search: the optimizer can revisit any promising intermediate result and explore alternative downstream decisions without re-running the expensive early stages.

3.2 Judge Agent: Decide to Navigate

As shown in Fig. 2, the Judge Agent is instantiated from a general-purpose LLM engine $\mathcal{L}$ by combining it with a system prompt $\mathcal{P}_J$ and a Harness Skill $\mathcal{U}_J$:

$$\text{Judge} = (\mathcal{L}, \mathcal{P}_J, \mathcal{U}_J), \quad (8)$$

where $\mathcal{P}_J$ is the system prompt that tells the Judge its role, defines the output schema (branching node, stage, and hints), and encodes design-specific search heuristics. $\mathcal{U}_J$ is the Harness Skill, which provides the Judge with two tools: an Observation Tool that constructs the search state profile and an Orchestrator Tool that dispatches decisions to downstream Stage Agents (both detailed in Section 3.4). At the beginning of iteration k , the harness provides the Judge with the optimization history $\mathcal{H}_k$ (Fig. 2), which contains the branching decisions $\mathcal{D}_i$ and per-stage QoR results from all previous iterations $i = 1, \dots, k-1$. This means every past iteration contributes to the current decision. Formally, the history accumulates one record per iteration:

$$\mathcal{H}_k = (\hat{n}_i, b_i, \{Q_i(s)\}_{s \geq b_i})_{i=1}^{k-1}, \quad (9)$$

where each entry records the branching node $\hat{n}_i$, the branching stage b_i , and the per-stage QoR $\{Q_i(s)\}_{s \geq b_i}$; the RT entry of the latter is the candidate’s post-route QoR. From this history, the Observation Tool computes an *adaptive profile*

$$\mathcal{A}_k = (\{E(n)\}_{n \in \mathcal{T}}, \{B(s)\}_{s \in \mathcal{S}}), \quad (10)$$

containing two signals, both computed by the Observation Tool (not by the Judge itself). The *exploration balance* $E(n)$ counts how often each node n has been branched from, so the Judge can identify over-explored and under-explored subtrees. The *stage bottleneck* $B(s)$ measures how far each stage’s recent QoR falls behind its best-known result, so the Judge can identify which stage is currently the weakest link. The Observation Tool assembles these signals together with the tree snapshot into a search state profile and delivers it to the Judge as its observation input.

Based on this search state profile, the Judge produces a decision:

$$\mathcal{D}_k = (\hat{n}_k, b_k, h_k), \quad (11)$$

where $\hat{n}_k$ is the node to branch from, $b_k \in \mathcal{S}$ is the branching stage, and $h_k = \{\text{hint}_s\}_{s \in \{b_k\} \cup \text{Aft}(b_k)}$ is a set of stage-specific hints, one per downstream Stage Agent (Section 3.4). The Judge selects b_k by targeting the stage with the largest bottleneck $B(s)$, and selects $\hat{n}_k$ by balancing QoR exploitation against exploration of less-visited nodes informed by $E(n)$. The decision is then dispatched to the Stage Agents, each of which executes its own stage-local action:

$$a_k(s) = \pi_s(\text{ctx}_s), \quad a_k(s) \in \Theta_s, \quad (12)$$

where ctx_s denotes the decision context available to the Stage Agent, detailed in Section 3.3.

After the Stage Agents finish executing from b_k onward, the tree is extended with the new nodes as defined in Equation (7), and the optimization history is appended:

$$\mathcal{H}_{k+1} = \mathcal{H}_k \cup (\hat{n}_k, b_k, \{Q_k(s)\}_{s \geq b_k}). \quad (13)$$

The global best candidate is updated by the timing-primary keep-the-best rule (Equation (18)). The updated $\mathcal{T}_{k+1}$ and $\mathcal{H}_{k+1}$ become the input at iteration $k+1$, closing the observe–decide–execute–feedback loop.

3.3 Stage Agents: Stage-Local Optimization

Each Stage Agent is similarly instantiated from the same LLM engine $\mathcal{L}$, equipped with a system prompt $\mathcal{P}_s$ and a PD Skill $\mathcal{U}_s$:

$$\text{StageAgent}_s = (\mathcal{L}, \mathcal{P}_s, \mathcal{U}_s), \quad (14)$$

where $\mathcal{P}_s$ is the system prompt that tells the agent which stage it is responsible for, what parameters it can tune, what design objectives to prioritize, and how to interpret stage-level feedback. $\mathcal{U}_s$ is the PD Skill for stage s . There are four PD Skills—FP Skill, PL Skill, CTS Skill, and RT Skill—each defining the available actions and execution interface for its corresponding stage (detailed in Section 3.4).

Once the Judge Agent decides to branch from node $\hat{n}_k$ at stage b_k (Equation (11)), the Stage Agents take over. For each stage $s \in \{b_k\} \cup \text{Aft}(b_k)$ executed in pipeline order, the harness first collects what the current agent needs to know: the QoR results from all preceding stages in the current branch (which come from the reused $\text{Bef}(b_k)$ checkpoint or from earlier Stage Agents in the same iteration), the Judge’s hint for this stage, and cross-iteration experience. These are assembled into a stage context:

$$\text{ctx}_s = (\{Q_k(i)\}_{i \in \text{Bef}(s)}, e_s, \text{hint}_s), \quad (15)$$

where $\{Q_k(i)\}_{i \in \text{Bef}(s)}$ is the upstream QoR from the current branch, e_s is the cross-iteration experience for stage s , and hint_s is the Judge’s stage-specific guidance.

With this context, the Stage Agent reasons over it, produces an action, executes it, and observes the result. One complete agentic step chains three operations: i) the agent decides the action $a_k(s) = \pi_s(\text{ctx}_s)$, ii) the harness executes the stage $\text{Execute}(s, a_k(s))$, iii) the backend reports the stage-level QoR $Q_k(s) = \text{Report}(s)$. The reported $Q_k(s)$ then becomes part of the upstream QoR for the next stage in the pipeline, so each Stage Agent builds on the results of the one before it.

3.4 Agent Harness

This subsection details the two skill modules shown in Fig. 2: the *Harness Skill* $\mathcal{U}_J$ that equips the Judge Agent, and the *PD Skill* $\mathcal{U}_s$ that equips each Stage Agent. Together they manage three responsibilities.

(1) *Observation Tool* (part of Harness Skill $\mathcal{U}_J$). At the beginning of iteration k , the Observation Tool constructs the search state profile that the Judge receives. It reads the optimization history $\mathcal{H}_k$ (Equation (9)) and the tree $\mathcal{T}_k$, computes the adaptive profile $\mathcal{A}_k$ (Equation (10)), and assembles a tiered tree snapshot that presents the most important nodes in full detail while compressing the rest. The resulting search state profile is delivered to the Judge as its

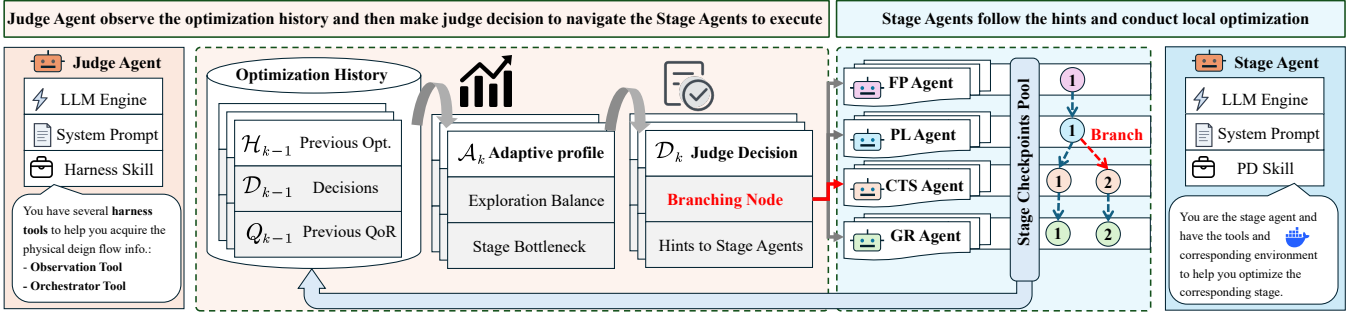


Figure 2: AgenticPD framework overview: the Judge Agent observes the optimization history and makes branching decisions, instructing the Stage Agents to execute each physical design stage from the selected branch node.

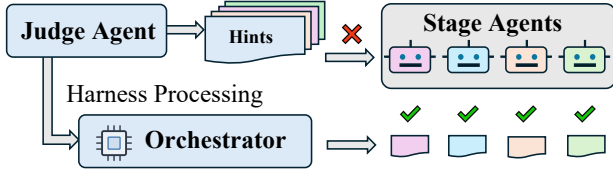


Figure 3: Orchestrator Harness: the Judge’s hints are dispatched to downstream Stage Agents, each of which executes through its own PD Skill.

observation input. This design also controls token usage: the Judge receives a compressed profile ($\mathcal{A}_k$ plus the tiered snapshot) rather than a raw history dump, and each Stage Agent only sees the fixed-size context ctx_s (Equation (15)), whose size does not grow with the number of iterations.

(2) *Orchestrator Tool* (part of Harness Skill $\mathcal{U}_f$). After the Judge produces the decision $\mathcal{D}_k = (\hat{n}_k, b_k, h_k)$ (Equation (11)), the Orchestrator Tool dispatches it, shown in Fig. 3. It calls the downstream Stage Agents one by one in pipeline order, delivering to each agent only its own hint. For example, if the Judge branches from CTS, the CTS agent receives the CTS hint and the RT agent receives the RT hint. The resulting action tuple for iteration k is therefore

$$\mathbf{a}_k = \left(\underbrace{a_k(s)}_{\text{reused from } \hat{n}_k}, s \in \text{Bef}(b_k), \underbrace{a_k(s)}_{\text{newly decided}}, s \in \{b_k\} \cup \text{Aft}(b_k) \right). \quad (16)$$

(3) *PD Skill* ($\mathcal{U}_s$). Each Stage Agent is equipped with its own PD Skill: one skill for each stage (FP Skill, PL Skill, CTS Skill, RT Skill), which gives the agent two things it needs to act. First, the PD Skill defines the available actions, allowed parameter ranges, and tunable knobs for that stage. The data format follows the IEEE CEDA METRICS2.1 standard [12], making AgenticPD compatible with the OpenROAD [1] ecosystem. Second, the PD Skill wraps the backend execution so that the agent can directly invoke the stage, launch the run, and receive the reported QoR $Q_k(s)$.

Together with the stage context ctx_s assembled from the Judge’s hint and upstream results (Equation (15)), the PD Skill lets each Stage Agent focus on its own stage without needing to access the tree or history directly. We illustrate this coordination with a concrete run in Section 4.5.

Algorithm 1: AgenticPD Workflow

Input: Design D , budget N , initial action tuple $\mathbf{a}_0$
Output: Best action tuple $\mathbf{a}^*$ and its post-route QoR Q^*

- 1 Run full flow under $\mathbf{a}_0$ through routing signoff, observe $\{Q_0(s)\}$ and Q_0
// Equations (3) and (5)
- 2 Init tree $\mathcal{T}$ with results;
- 3 $Q^* = Q_0$;
- 4 $\mathcal{H} = \emptyset$; // Equations (6) and (9)
- 5 **for** $k = 1$ to N **do**
- 6 $\mathcal{A}_k = \text{OBSERVATIONTOOL}(\mathcal{T}, \mathcal{H})$; // adaptive profile
from Equation (10)
- 7 $(\hat{n}_k, b_k, h_k) = \text{JUDGE}(\mathcal{H}, \mathcal{A}_k)$; // decision
from Equation (11)
- 8 Reuse $\text{Bef}(b_k)$ from node $\hat{n}_k$;
- 9 **for** each stage $s \in \{b_k\} \cup \text{Aft}(b_k)$ **do**
- 10 $ctx_s = \text{BUILDCONTEXT}(s, \hat{n}_k, \text{hint}_s)$; // Equation (15)
- 11 $a_k(s) = \text{STAGEAGENT}_s(ctx_s)$; // Equation (12)
- 12 $Q_k(s) = \text{EXECUTESTAGE}(s, a_k(s))$; // Section 3.3
- 13 Update $\mathcal{T}$ and $\mathcal{H}$; // Equations (7) and (13)
- 14 Update $(\mathbf{a}^*, Q^*)$ if $v_k \succ v^*$; // Equation (18)
- 15 **Report** $(\mathbf{a}^*, Q^*)$;

3.5 Overall Algorithm

This subsection summarizes the AgenticPD workflow with the multi-agent and harness system.

Lines 1–4 initialize the search: run the full flow under $\mathbf{a}_0$ through the routing signoff, record the results in the tree $\mathcal{T}$ and history $\mathcal{H}$. Lines 6–14 describe one outer-loop iteration. The Observation Tool first constructs the adaptive profile $\mathcal{A}_k$ from the current tree and history (Line 6). The Judge then produces the branching decision $\mathcal{D}_k$ (Line 7). The system reuses $\text{Bef}(b_k)$ from node $\hat{n}_k$ (Line 8) and re-executes $\{b_k\} \cup \text{Aft}(b_k)$ (Lines 9–12): for each stage s , the harness builds ctx_s , and the Stage Agent decides $a_k(s)$ and executes the stage. Lines 13–14 update the tree, history, and global best.

Because $Q(s)$ is a multi-dimensional tuple (Section 2) and our objective is timing-first with power and area as guardrails (Section 4.1), we keep the best candidate by a timing-primary rule rather than by a single scalar. Let $Q(v)$ denote the QoR tuple of a completed node $v \in \mathcal{T}$, measured at the post-route signoff, and let Q_1 denote its primary timing metric. We say u dominates v when it is at least

Table 1: AgenticPD vs. Vanilla LLM on ASAP7. Bold = AgenticPD is the best.

Method	Design	Qwen3.5 [18]				Kimi-K2.5 [14]				DeepSeek-V3.2 [5]			
		WNS $\uparrow$	TNS $\uparrow$	Area $\downarrow$	Power $\downarrow$	WNS $\uparrow$	TNS $\uparrow$	Area $\downarrow$	Power $\downarrow$	WNS $\uparrow$	TNS $\uparrow$	Area $\downarrow$	Power $\downarrow$
Vanilla LLM	AES	-50.749	-2958.5	2.1	0.159	-39.768	-2085.8	2.1	0.159	-38.338	-2278.2	2.1	0.158
	ibex	-66.140	-3325.2	2.7	0.048	-66.140	-3325.2	2.7	0.048	-66.140	-3325.2	2.7	0.048
	JPEG	10.646	0	6.8	0.095	-5.006	-7.5	6.7	0.092	-5.006	-7.5	6.7	0.092
AgenticPD	AES	-35.457	-2654.4	2.0	0.157	-40.064	-2561.0	2.0	0.158	-32.271	-1811.3	2.0	0.157
	ibex	-99.135	-11271.5	2.6	0.046	-65.416	-1837.8	2.7	0.048	-53.867	-1098.7	2.7	0.048
	JPEG	55.915	0	6.7	0.092	56.711	0	6.7	0.092	50.569	0	6.7	0.092
Geomean impr.		AgenticPD better in all PPA: Performance 2.4%, Area 2.2%, Power 1.1%											

^{*}WNS and TNS are reported in picoseconds, where higher (less negative) is better ($\uparrow$). Area is in $10^3 \mu\text{m}^2$ and power in watts, where lower is better ($\downarrow$). Geomean improvement is computed as the geometric mean of per-case ratios (AgenticPD / Vanilla LLM), with performance measured by effective clock period (ECP = clock period - WNS), the metric is computed following [8].

as good on every dimension and strictly better on at least one:

$$u \succ v \iff Q_j(u) \geq Q_j(v) \quad \forall j, \quad Q(u) \neq Q(v). \quad (17)$$

Let v^* denote the current incumbent. The incumbent is a reporting choice rather than a search driver: it advances whenever a completed candidate v_k improves the primary metric Q_1 while keeping the guardrail metrics within tolerance, with Pareto dominance (Equation (17)) being the special case where every dimension improves:

$$(a^*, Q^*) = \begin{cases} (a_k, Q_k) & \text{if } v_k \text{ improves } Q_1 \text{ within the guardrails,} \\ (a^*, Q^*) & \text{otherwise.} \end{cases} \quad (18)$$

Because the update keys on the primary metric, the incumbent keeps advancing through the search instead of stalling when no candidate improves every dimension at once, and it does not restrict the branching exploration, which continues over the full tree. Every completed candidate is measured at the post-route signoff, so the incumbent's QoR is reported directly, with no separate final evaluation step.

4 Experiments

4.1 Experimental Setup

All experiments were conducted on a Linux workstation running Ubuntu 22.04 LTS, equipped with dual Intel Xeon Gold 6426Y processors and 256 GB RAM. AgenticPD and all baselines share the same backend: Standard Docker environment integrated with OpenROAD [1] denoted by OpenROAD-flow-scripts [23], which means every method has the same tuning parameters and exploration space.

We evaluate three designs spanning diverse functional categories: AES [16], ibex [19], and JPEG [17] on both tech nodes: the SkyWater 130 nm HD library [9] and the ASAP 7 nm predictive PDK [4], with a 20-iteration budget for backend runs. All methods are configured to optimize post-route QoR, with timing as the primary metric. All LLM-based methods run at temperature 0.6; given fixed model outputs the orchestration is deterministic, and the ORFS backend is deterministic per configuration, so a given configuration reproduces bit-identical QoR. Following the stage formulation in Section 2, all methods start from the post-synthesis netlist and execute floorplan, placement, CTS, and routing under the same backend during iterative optimization. Every candidate is carried through detailed routing, so the QoR observed in the loop and the QoR reported in the tables are the same post-route signoff metrics; no proxy is used at any point.

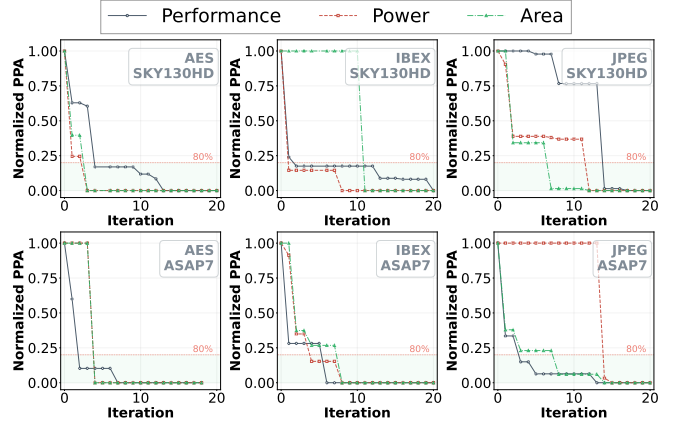


Figure 4: Normalized PPA convergence curves.

4.2 AgenticPD vs. Vanilla LLM

To evaluate whether the agentic architecture itself drives the improvement, we first compare AgenticPD against a Vanilla LLM baseline, a flat single agent that shares the same agent-friendly environment integration as AgenticPD. It uses the same 20-evaluation budget and Docker backend. A single LLM proposes one complete parameter configuration per iteration, seeing full evaluation history. We compare the two systems on the ASAP7 technology node with three LLM engines: DeepSeek-V3.2 [5], Qwen3.5 [18], and Kimi-K2.5 [14].

TABLE 1 reports post-route PPA on three ASAP7 benchmarks with three LLM backbones. AgenticPD achieves better timing than Vanilla LLM in 7 of the 9 pairs. The gains are largest on the hard cases: on JPEG ASAP7, AgenticPD reaches positive post-route WNS (timing closure) across all three backbones, while Vanilla LLM either fails or only barely closes. These results indicate that, in our evaluation, the multi-agent architecture outperforms a monolithic LLM agent on most of these cases.

Among the three backbones, DeepSeek-V3.2 delivers the best overall timing, so we adopt it as the default for the remaining experiments when the method needs the LLM engine.

4.3 Convergence and Branching Analysis

With DeepSeek-V3.2 as the LLM engine, we run AgenticPD on all six benchmarks described in Section 4.1 and analyze its convergence behavior and branching decisions.

Table 2: Post-route PPA comparison across six benchmarks. Bold = AgenticPD is the best.

Tech	Design	OpenROAD [1, 23]				AutoTuner [12]				ORFS-Agent [8]				AgenticPD (Ours)			
		WNS $\uparrow$	TNS $\uparrow$	Area $\downarrow$	Power $\downarrow$	WNS $\uparrow$	TNS $\uparrow$	Area $\downarrow$	Power $\downarrow$	WNS $\uparrow$	TNS $\uparrow$	Area $\downarrow$	Power $\downarrow$	WNS $\uparrow$	TNS $\uparrow$	Area $\downarrow$	Power $\downarrow$
SKY130HD	AES	-0.321	-2.2	125.2	0.424	0.133	0	123.7	0.424	0.120	0	126.8	0.432	0.271	0	127.8	0.437
	ibex	-0.664	-23.8	181.0	0.110	-0.719	-56.9	183.3	0.112	-0.581	-22.2	181.6	0.110	-0.503	-19.3	179.4	0.107
	JPEG	-0.218	-2.8	516.3	0.542	-0.207	-4.0	511.1	0.546	-0.202	-3.6	519.6	0.549	-0.154	-1.6	517.7	0.531
ASAP7	AES	-37.614	-2395.2	2.1	0.157	-43.399	-2392.4	2.0	0.157	-37.444	-2566.4	2.1	0.158	-32.271	-1811.3	2.0	0.157
	ibex	-62.655	-2740.9	2.7	0.047	-61.852	-3145.1	2.7	0.048	-63.669	-1057.6	2.7	0.047	-53.867	-1098.7	2.7	0.048
	JPEG	-8.229	-37.4	6.7	0.092	-9.996	-25.9	6.7	0.092	0.337	0	6.7	0.092	50.569	0	6.7	0.092

* OpenROAD = default parameters from the bundled OpenROAD-flow-scripts configuration. WNS and TNS are reported in picoseconds for ASAP7 and nanoseconds for SKY130HD, where higher (less negative) is better ($\uparrow$). Area is in $10^3 \mu m^2$ and power in watts, where lower is better ($\downarrow$).

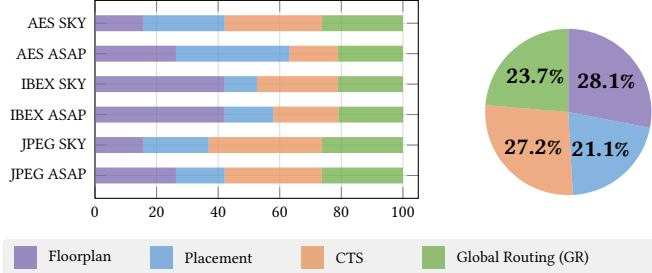


Figure 5: Distribution of branching stages across all AgenticPD sessions. Left: per-benchmark breakdown. Right: aggregate distribution over all benchmarks.

Convergence analysis Fig. 4 shows how the in-loop post-route QoR evolves during a single optimization run. For each benchmark, we plot the normalized best-so-far gap for ECP [8], power, and area: a gap of 1 corresponds to the initial configuration and 0 to the best configuration found. Across all six benchmarks, all three objectives converge to within 20% of their final values by iteration 8–14, with most benchmarks crossing the 80% convergence line (dashed red) before the midpoint of the budget. Once below this line, the best-so-far configuration is already close to the final in-loop optimum, and subsequent iterations yield only incremental refinement. Crucially, the three curves decrease largely in tandem rather than trading off, so within these runs AgenticPD improves timing without substantially sacrificing area or power. This fast convergence also means the iteration budget N stays small in practice. Because the compressed observation profile delivered to the Judge (Section 3.4) grows slowly with N and each Stage Agent sees only a modest context ctx_s (Equation (15)), the peak prompt across all agents and benchmarks reaches only 43K tokens, well within the 128K [5], 256k [14] and 1000k [18] context window of the modern LLMs.

This joint convergence stems from the stage-aware decomposition. The FP agent drives the area down by exploring utilization. The CTS agent tunes clock-tree parameters that jointly affect power and timing. The Judge coordinates exploration so that improvements in one objective do not regress others. For AES on ASAP7, area is nearly constant across iterations (the dash-dotted line overlaps the x-axis); this is consistent with TABLEs 1 and 2, where all methods converge to similar area values for this design.

Branch stage analysis Fig. 5 shows the distribution of branching stages selected by the Judge across all AgenticPD sessions. The four stages are relatively balanced: FP is slightly most frequent at 28.1%, followed by CTS (27.2%), RT (23.7%), and PL (21.1%). This reflects the Judge’s diagnostic strategy: when utilization is the bottleneck,

floorplan branches rebuild the entire downstream pipeline; CTS and RT branches preserve proven upstream stages and target clock-tree repair or routing congestion, respectively; and placement branches adjust global-placement parameters such as timing- and routability-driven modes. This suggests that no single stage dominates the optimization landscape and that the Judge uses stage-level branching to address diverse bottlenecks. Moreover, because a branch at stage s reuses the cached $Bef(s)$ results instead of recomputing them, AgenticPD executes fewer backend stage runs than full-flow tuners at the same iteration budget.

4.4 AgenticPD vs. Previous Tuners

We next compare AgenticPD against previous state-of-the-art tuners that can be applied to PD optimization problems: AutoTuner [12] as the representative black-box tuner and ORFS-Agent [8] as a representative LLM-based tuner. Both AgenticPD and ORFS-Agent use DeepSeek-V3.2 as the LLM engine. We also include the default OpenROAD-flow-scripts configuration as a no-tuning reference.

As shown in TABLE 2, AgenticPD achieves the best post-route WNS among the compared methods on all six benchmarks. On JPEG ASAP7, AgenticPD reaches timing closure (WNS = +50.6 ps) where AutoTuner does not (WNS = -10.0 ps) and ORFS-Agent is marginal (WNS = +0.3 ps). On AES ASAP7, a hard timing case, AgenticPD improves WNS by 11.1 ps over AutoTuner and 5.2 ps over ORFS-Agent. AgenticPD generally avoids trading physical quality for timing: on most benchmarks it stays competitive in area and power, although it does not dominate every method on these secondary metrics. AutoTuner [12] and ORFS-Agent [8] both treat the parameter space as flat Θ_{PD} and restart the full flow every trial, without diagnosing which stage causes a regression or preserving upstream progress. AgenticPD instead diagnoses the bottleneck stage and lets each Stage Agent reason within its own Θ_s with stage-level feedback. This is not LLM prompt engineering applied to tuning; it is a purpose-built agentic system whose architecture encodes the structure of the PD optimization problem.

4.5 Case Study: JPEG on ASAP7

Fig. 6 compares the OpenROAD layout with the AgenticPD result on JPEG (ASAP7); AgenticPD improves post-route WNS from -8.2 ps to +50.6 ps.

To illustrate how the multi-agent system reaches this result, we trace its optimization trajectory. The Judge first directs exploration toward high core utilization: the FP Agent sets CU = 45 and the CTS Agent selects an aggressive clock-tree configuration (CTS_CLUSTER_SIZE = 60,

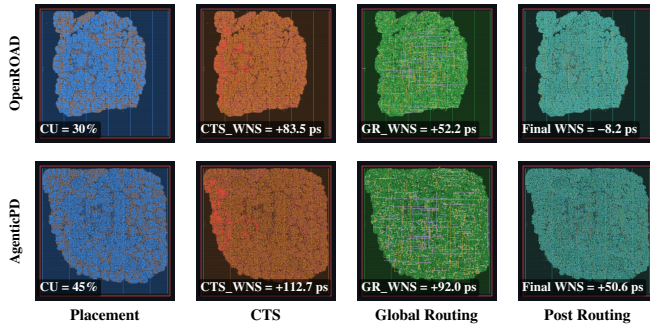


Figure 6: Layout comparison for the JPEG design on ASAP7, shown at the placement, CTS, global routing, and post-routing stages. AgenticPD improves post-route WNS from -8.2 ps to $+50.6$ ps.

SLACK_MARGIN = 0.15, TNS% = 10), establishing a strong floorplan–placement foundation. A later floorplan branch lowers utilization to CU = 44, but this rebuilds the entire downstream pipeline and regresses timing—revealing that even a 1% utilization change can disrupt a well-optimized placement–CTS configuration. The Judge reflects on this regression and shifts strategy: instead of re-exploring floorplan, it branches from the earlier node at the CTS stage, preserving the proven FP and PL stages, and the CTS Agent retunes to a balanced setting (CTS_CLUSTER_SIZE = 45, SLACK_MARGIN = 0.13, TNS% = 35), reaching the session-best post-route WNS = $+50.6$ ps without re-running any upstream stage. This trajectory illustrates the two core mechanisms: the Judge diagnoses bottlenecks and selects the right branch point, while Stage Agents autonomously tune parameters within their scope.

5 Conclusion

In this paper, we presented AgenticPD, a stage-aware agentic framework that organizes physical design QoR optimization as a tree search with branching, where a Judge Agent navigates the search and stage-specialized agents make local decisions. In our evaluation, AgenticPD outperforms both vanilla LLM baselines and prior state-of-the-art physical design tuning methods on timing. Across six benchmarks on two technology nodes, it achieves the best post-route timing among the compared methods while remaining competitive in power and area.

References

- [1] Tutu Ajayi, Vidya A Chhabria, Mateus Fogaça, Soheil Hashemi, Abdelrahman Hosny, Andrew B Kahng, Minsoo Kim, Jeongsup Lee, Uday Mallappa, Marina Neseem, et al. 2019. Toward an open-source digital flow: First learnings from the OpenROAD project. In *Proceedings of the Design Automation Conference (DAC)*. 1–4.
- [2] Anthropic. 2024. Building Effective Agents. Anthropic Research Blog. <https://www.anthropic.com/research/building-effective-agents>
- [3] Cadence Design Systems. 2025. Cadence Welcomes ChipStack. https://community.cadence.com/cadence_blogs_8/b/corporate-news/posts/cadence-welcomes-chipstack.
- [4] Lawrence T Clark, Vinay Vashishtha, Lucian Shifren, Aditya Gujja, Saurabh Sinha, Brian Cline, Chandrasekaran Ramamurthy, and Greg Yeric. 2016. ASAP7: A 7-nm finFET predictive process design kit. *Microelectronics Journal* 53 (2016), 105–115.
- [5] DeepSeek. 2025. *DeepSeek-V3.2 Release*. <https://api-docs.deepseek.com/news/news251201>
- [6] Hao Geng, Tinghuan Chen, Yuzhe Ma, Binwu Zhu, and Bei Yu. 2022. PTPT: Physical design tool parameter tuning via multi-objective Bayesian optimization. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 1 (2022), 178–189.

- [7] Hao Geng, Qi Xu, Tsung-Yi Ho, and Bei Yu. 2022. PPATuner: Pareto-driven tool parameter auto-tuning in physical design via Gaussian process transfer learning. In *Proceedings of the ACM/IEEE Design Automation Conference (DAC)*. 1237–1242.
- [8] Amur Ghose, Andrew B Kahng, Sayak Kundu, and Zhiang Wang. 2025. ORFS-Agent: Tool-using agents for chip design optimization. In *Proceedings of the ACM/IEEE Symposium on Machine Learning for CAD (MLCAD)*. IEEE, 1–13.
- [9] Google and SkyWater Technology. 2020. SkyWater SKY130 Open Source PDK. <https://github.com/google/skywater-pdk>. Accessed: 2026-03-20.
- [10] Hao-Hsiang Hsiao, Yi-Chen Lu, Pruek Vanna-Iampikul, and Sung Kyu Lim. 2024. FastTuner: Transferable physical design parameter optimization using fast reinforcement learning. In *Proceedings of the International Symposium on Physical Design (ISPD)*. 93–101.
- [11] Hao-Hsiang Hsiao, Pruek Vanna-Iampikul, Yi-Chen Lu, and Sung Kyu Lim. 2024. ML-based Physical Design Parameter Optimization for 3D ICs: From Parameter Selection to Optimization. In *Proceedings of the ACM/IEEE Design Automation Conference (DAC)*. Article 252, 6 pages. doi:10.1145/3649329.3656509
- [12] Jinwook Jung, Andrew B Kahng, Seungwon Kim, and Ravi Varadarajan. 2021. METRICS2.1 and flow tuning in the IEEE CEDA robust design flow and OpenROAD ICCAD special session paper. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 1–9.
- [13] Andrew B Kahng, Jens Lienig, Igor L Markov, and Jin Hu. 2011. *VLSI physical design: from graph partitioning to timing closure*. Vol. 312. Springer.
- [14] Moonshot AI. 2026. *Kimi-K2.5 Model by Moonshotai | NVIDIA NIM*. <https://build.nvidia.com/moonshotai/kimi-k2.5/modelcard>
- [15] OpenAI. 2026. *ChatGPT*. <https://openai.com/research>
- [16] OpenCores. 2002. AES (Rijndael) IP Core. https://opencores.org/projects/aes_core. Accessed: 2026-03-20.
- [17] OpenCores. 2002. JPEG Encoder, Video Compression Systems Project. https://opencores.org/projects/video_systems. Accessed: 2026-03-20.
- [18] Qwen Team. 2026. *Qwen3.5: Towards Native Multimodal Agents*. <https://qwen.ai/blog?id=qwen3.5>
- [19] Pasquale Davide Schiavone, Francesco Conti, Davide Rossi, Michael Gautschi, Antonio Pullini, Eric Flamand, and Luca Benini. 2017. Slow and steady wins the race? A comparison of ultra-low-power RISC-V cores for Internet-of-Things applications. In *Proceedings of the International Symposium on Power and Timing Modeling, Optimization and Simulation (PATMOS)*. IEEE, 1–8.
- [20] Utsav Sharma, Bing-Yue Wu, Sai Rahul Dhanvi Kankipati, Vidya A Chhabria, and Austin Rovinski. 2024. OpenROAD-Assistant: An open-source large language model for physical design tasks. In *Proceedings of the ACM/IEEE International Symposium on Machine Learning for CAD (MLCAD)*. 1–7.
- [21] Siemens Digital Industries Software. 2026. Siemens launches Fuse EDA AI Agent. <https://news.siemens.com/en-us/siemens-fuse-eda-ai-agent/>.
- [22] Synopsys. 2026. Agentic AI: Automating Engineering Workflows with AI Agents. <https://www.synopsys.com/ai/agentic-ai.html>.
- [23] The OpenROAD Project. 2024. OpenROAD-flow-scripts: Autonomous Digital Design Flow. <https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts>. Accessed: 2026-03-20.
- [24] Bing-Yue Wu, Utsav Sharma, Austin Rovinski, and Vidya A Chhabria. 2025. OpenROAD Agent: An Intelligent Self-Correcting Script Generator for OpenROAD. In *Proceedings of the IEEE International Conference on LLM-Aided Design (ICLAD)*. IEEE, 16–22.
- [25] Haoyuan Wu, Zhuolun He, Xinyun Zhang, Xufeng Yao, Su Zheng, Haisheng Zheng, and Bei Yu. 2024. ChatEDA: A large language model powered autonomous agent for EDA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 10 (2024), 3184–3197.
- [26] Su Zheng, Hao Geng, Chen Bai, Bei Yu, and Martin DF Wong. 2023. Boosting VLSI design flow parameter tuning with random embedding and multi-objective trust-region Bayesian optimization. *ACM Transactions on Design Automation of Electronic Systems* 28, 5 (2023), 1–23.